AgroLink Hub

Connect. Share. Sell. Grow Together.

FINAL PROJECT REPORT

AgroLink Hub

Social commerce for local producers, small businesses, buyers, and community learning.

A full-stack platform where people can share useful updates, discover trusted sellers, chat with confidence, publish products, manage orders, and grow a healthier local digital marketplace.

Frontend
React 18, Vite, Axios, React Router, STOMP client.

Backend
Spring Boot 4, Spring Security, JPA, WebSocket, Mail, OAuth2.

Database
PostgreSQL with schema patching and JPA-managed entities.

Prepared on June 19, 2026.

CONTENTS

Report Overview

This document explains the product idea, system structure, technology choices, and main workflows in a way that a reviewer, lecturer, developer, or project owner can understand without needing to inspect every source file first.

1. Project summary and product purpose
2. Technology stack and dependencies
3. High-level architecture
4. Repository and folder structure
5. Frontend application modules
6. Backend application modules
7. Database model and persistence
8. Authentication, roles, and route security
9. Social media feature set
10. Business and marketplace feature set
11. Chat, notifications, support, and assistant
12. Build, testing, deployment, and final notes

Current product polish included: The landing page now presents AgroLink as a social marketplace, the feature cards use generated Social, Commerce, and Workspace visuals, marketplace/cart/order screens follow the AgroLink palette, and notifications are kept compact enough for daily use. Messaging includes stable composer behavior, reactions, online presence, and sent/delivered/read tick feedback.

PRODUCT

What AgroLink Hub Is

AgroLink Hub is a combined social networking and marketplace application. It is designed for everyday users, creators, small businesses, and farmers who need one platform to share posts, build relationships, sell products, manage orders, communicate with buyers, and receive platform support.

Social networking

Users can create posts, react, comment, reply, share, save posts, view stories, follow users, manage friends, and receive notifications.

Marketplace and commerce

Business and farmer sellers can publish products. Buyers can browse marketplace products, add items to cart, checkout, and track orders.

Account workspace

Users get profile pages, settings, calendar planning, support tickets, account verification, password reset, and onboarding.

Admin management

Admin users can review users, moderate reports, manage support workflows, and monitor platform activity.

Target users

- Regular users who want a community feed and marketplace shopping.
- Business sellers who want product listings, orders, and analytics.
- Farmer sellers who want to sell produce or livestock-related products.
- Creators who want social reach and marketplace discovery.
- Admins who need moderation, support, and user-management tools.

Why this model is useful

A producer can reach buyers directly instead of depending only on intermediaries. That can help customers pay a fairer price while giving farmers and small sellers a better return. The social layer also matters: posts, comments, stories, follows, reviews, and chat make sellers feel real, not just product cards in a catalogue.

Friends and communities sharing posts, stories, and conversations.

Fresh products, cart actions, orders, and seller-to-buyer commerce.

Calendar, analytics, support, admin, and seller workspace tools.

Core value

The project joins community engagement and commerce in one workspace. A user can discover people, message them, buy products, review businesses, follow order status, and ask for support without leaving the platform.

AgroLink Hub product scope

STACK

Frameworks, Libraries, and Runtime Tools

Layer	Technology	Purpose
Frontend	React 18	Builds the user interface with component-based pages and shared dashboard UI components.
Frontend	Vite 6	Provides local dev server, asset bundling, and production build output.
Routing	React Router DOM 6	Handles public routes, protected routes, nested app shell routes, and role-based routes.
HTTP	Axios	Central API client with credential support and refresh handling.
Realtime	@stomp/stompjs	Frontend WebSocket/STOMP support for chat and realtime features.
Backend	Spring Boot 4.0.2	Main Java backend framework for REST APIs, security, data access, mail, and WebSocket.
Security	Spring Security, OAuth2 Client, JJWT	Login, JWT cookies/headers, Google OAuth2, role protection, and authenticated APIs.
Persistence	Spring Data JPA, PostgreSQL	Entity mapping, repositories, and relational storage.
Validation	Spring Validation	Validates incoming backend request payloads.
Similarity	Apache Commons Text	Used by the chatbot knowledge service for fuzzy matching.

Runtime configuration

The backend reads settings from `application.yaml` and optional `.env` properties. Important environment values include database connection, mail credentials, JWT secret, cookie behavior, OAuth2 redirect URLs, CORS origins, server port, and file upload path.

Build commands

- Frontend build: `npm run build` inside `Lisharefrontend`.
- Backend build: Maven wrapper or local Maven command inside `Lisharebackend`.
- Local frontend dev server: Vite on `http://localhost:5173`.
- Backend API server: Spring Boot on `http://localhost:8081` by default.

AgroLink Hub technology stack

ARCHITECTURE

High-Level System Design

The codebase is split into a React frontend and Spring Boot backend. Both sides follow a similar domain layout: `platform`, `social`, and `business`. This makes it easier to understand which parts serve accounts, social features, and commerce workflows.

Client application

React renders public auth pages, landing page, protected app shell pages, dashboard navigation, marketplace, cart, orders, chat, feed, profile, support, calendar, analytics, and admin screens.

API gateway style backend

Spring controllers expose REST endpoints. Services contain business rules. Repositories persist data through JPA. Security filters protect authenticated APIs.

Realtime messaging

WebSocket STOMP is configured at `/ws` with `/topic`, `/queue`, and `/user` destinations.

Media delivery

Uploaded files are served from `/uploads/**` through a configured Spring MVC resource handler.

Request flow

- The browser sends requests through Axios to the backend base URL.
- Cookies are included because the Axios instance uses `withCredentials: true`.
- Spring Security reads JWT from the Authorization header or cookies.
- Controllers delegate to services, services call repositories, and repositories access PostgreSQL.
- Responses return DTOs or standard response objects consumed by React pages.

Frontend route flow

Public pages include landing, login, signup, verify OTP, forgot password, and OAuth callback. Authenticated pages are nested under `ProtectedRoute` and rendered inside `AppShell`. Seller and admin pages are further protected by `RoleRoute`.

AgroLink Hub architecture overview

STRUCTURE

Repository and Folder Structure

Path	Meaning
`Lisharefrontend`	React/Vite application, assets, modules, shared UI, CSS, services, and route definitions.
`Lisharebackend`	Spring Boot API application with controllers, services, repositories, entities, security, configuration, and schema SQL.
`uploads`	Local project-level storage used for uploaded media during development.
`docs`	Project documentation folder created for this report and generated PDF.

Frontend module layout

- `modules/platform` : auth, app store/routes, common UI, layouts, user profile/settings, support, calendar.
- `modules/social` : posts/feed, comments, stories, friends, chat, notifications, chatbot assistant services.
- `modules/business` : marketplace, products, cart, orders, business pages, reviews, support, analytics, admin.
- `assets` : branding logos, backgrounds, commerce images, and UI visual assets.
- `dashboard-ui.css` : the main visual system stylesheet for dashboard, auth, commerce, social, and responsive behavior.

Backend module layout

- `modules/platform` : auth, users, calendar, security, storage, config, common enums/utilities.
- `modules/social` : posts, comments, reactions, shares, stories, friends, follow, chat, notifications, chatbot.
- `modules/business` : business pages, products, cart, orders, reviews, support, admin services.
- `src/main/resources` : application YAML and SQL schema patches.

> The mirrored frontend/backend domain structure is one of the strongest maintainability decisions in the project. It lets developers trace a feature from React page to API endpoint to service and entity.

AgroLink Hub folder structure

FRONTEND

Frontend Pages and UI Behavior

The frontend is a single-page application. It uses public routes for authentication and a protected app shell for logged-in workflows. The app shell supplies navigation, user context, page metadata, notifications, and consistent layout patterns.

Area	Pages	Purpose
Auth	Landing, login, signup, verify OTP, forgot password, OAuth callback, onboarding	Account creation, verification, session start, recovery, and first-time setup.
Social	Home feed, profile, friends, chat, notifications	Community feed, user identity, relationships, realtime messaging, and alerts.
Commerce	Marketplace, cart, orders, business page, analytics	Product browsing, shopping, order tracking, seller publishing, and insights.
Platform	Calendar, support, settings	Planning, help desk, account media, email/password changes, and user preferences.
Admin	Admin dashboard and admin subroutes	User management, role workflows, support tickets, and moderation.

Design system notes

- Shared UI components live in `DashboardUI.jsx`: buttons, cards, modals, status badges, avatars, stat cards, tabs, and icons.
- The primary visual palette now follows AgroLink green, yellow, teal, dark green brand surfaces, and restrained red only for destructive actions.
- The login page uses the same auth design language as sign-up, but with a separate auth-collage background image.
- The cart page uses matching commerce colors and removes the earlier purple/blue checkout mismatch.
- The notification dropdown is compacted so it does not cover too much of the page.

API usage

All endpoint constants are centralized in `modules/platform/api/endpoints.js`. Services import these constants and the shared Axios instance, which keeps frontend API paths consistent across features.

BACKEND

Backend Modules and API Responsibilities

The backend is a Spring Boot application with controllers, services, repositories, entities, security filters, WebSocket configuration, mail support, and schema patching.

Module	Main responsibilities
`platform/auth`	Register, login, refresh, logout, OTP verification, forgot password, Google OAuth2.
`platform/user`	User profile, account updates, profile media, cover media, blocking users, public profile data.
`platform/security`	Security filter chain, JWT authentication filter, authentication provider, route permissions.
`social/post`	Posts, feed, saved posts, reports, poll voting, media fields, moderation support.
`social/comment`	Comments, replies, comment media, updates, deletion, comment reactions.
`social/story`	Temporary story media, views, reactions, replies, sharing, expiry behavior.
`social/chat`	Direct conversations, group creation UI and APIs, messages, image/video attachments, reactions, seen state, presence, typing events, and WebSocket messaging.
`business/product`	Marketplace product creation, updates, public browsing, product availability and stock.
`business/cart`	Cart item persistence, quantity update, removal, clearing, checkout into orders.
`business/order`	Buyer orders, seller received orders, cancellation, and status updates.
`business/admin`	Admin user management, moderation status, stats, reports.

Controller inventory

The backend contains controllers for admin reports/users, auth, forgot password, calendar, users, user blocks, chat, chatbot, comments, follow, friends, notifications, posts, post safety, reactions, shares, stories, products, business pages, cart, orders, reviews, and support.

Service layer

Services contain workflow logic such as account registration, cart checkout, order creation, business page validation, product management, review handling, support replies, post and story behavior, chat conversations, and notification creation.

PERSISTENCE

Database Model and Schema Behavior

The database is PostgreSQL. Spring Data JPA maps Java entities to tables, and `schema.sql` plus `DatabaseSchemaPatch` keep existing local databases compatible as features are added.

Main entity families

Users and auth

User, forgot-password records, roles, OTP fields, profile media, account metadata, and blocks.

Social graph

Follow, friend requests, notifications, saved posts, post reports, stories, story views, and story reactions.

Content

Posts, poll votes, comments, comment reactions, reactions, shares, story media, and media URL fields.

Commerce

Business pages, products, cart items, orders, reviews, support questions, and order statuses.

Chat

Direct chats, group creation support, members, messages, seen state, attachments, presence, typing events, and message reactions.

Assistant

Concepts table with topic, keywords, and description for knowledge-base answers.

Schema patch examples

- Post media fields, poll fields, audience, edited time, feeling, and location metadata.
- Comment media fields and comment reaction tables.
- Story tables, story reactions, story views, and reshare metadata.
- Product delivery method, cart items, order delivery fields, reviews, and support structures.
- Chatbot concepts table with searchable keywords and answer descriptions.

Storage behavior

The backend exposes uploaded files through `/uploads/**`. Multipart upload limits are configured to support larger media files, with defaults of 100 MB per file and 120 MB per request.

AgroLink Hub database and persistence

ROLES

Authentication, Authorization, and RLS-Style Checks

The project does not use database-native PostgreSQL row-level security policies. Instead, access control is implemented at application level through Spring Security, controller/service checks, frontend protected routes, and role-specific navigation. For this project, "RLS check" should be understood as role-level security and resource-level access enforcement in the application layer.

Authentication methods

- Email/password login through backend auth endpoints.
- Google OAuth2 login with configured success and failure redirects.
- JWT access and refresh tokens read from Authorization headers or cookies.
- OTP verification after signup and password reset flows.
- Session refresh endpoint used by the React auth store to keep users logged in.

Backend security controls

- Spring Security permits public auth, forgot-password, OAuth2, WebSocket, reviews, uploads, public products, public business pages, and assistant chat.
- Admin APIs under `/api/admin/**` require the admin role.
- All other API requests require authentication.
- CORS allows configured local frontend origins and credentials.
- JWTAuthFilter normalizes OAuth2 session users into application user details when possible.

Frontend role controls

- `ProtectedRoute` blocks logged-out users from app shell pages.
- `RoleRoute` blocks non-seller users from seller tools and non-admin users from admin tools.
- Navigation filters items based on roles such as `ROLE_USER`, `ROLE_CREATOR`, `ROLE_BUSINESS`, `ROLE_FARMER`, and `ROLE_ADMIN`.

> Security note: the implemented scope separates public pages, authenticated app pages, seller tools, and admin tools through backend role checks and frontend route guards, so sensitive workflows stay behind the correct user permissions.

SOCIAL

Social Media Feature Set

The social side keeps the platform active between purchases. Users can post updates, react, comment, follow people, chat, and receive notifications, while sellers use the same community attention to build trust before a buyer places an order.

Feed and posts

Supports feed loading, creating posts, media URLs, audience metadata, poll questions/options, saved posts, sharing, reports, and moderation support.

Comments and reactions

Users can comment, reply, update comments, delete comments, react to posts, and react to comments.

Stories

Users can create temporary media stories, view story feeds, react, reply, share, track views, and delete stories.

Friends and following

The app supports search, follow/unfollow, follower/following counts, friend requests, accepts, rejects, cancels, and unfriending.

Profile

Profile pages show identity, media, posts, saved items, and account actions. Settings support profile and cover image management.

Notifications

Notification APIs provide list, unread count, mark-read, read-all, and clear-all behavior. The dropdown is now compact to avoid covering too much of the page.

Realtime and chat

The chat module contains direct conversation UI, group creation UI/API support, message APIs, sent/delivered/read ticks, members, image/video attachments, reactions, online/offline presence, typing events, and WebSocket delivery. Blue read ticks help users understand whether a message has reached the other person and whether it has been seen, which is important for buyer-seller conversations about stock, price, delivery, and orders. Pinned link persistence is outside the final project scope; completed chat behavior focuses on reliable messaging, media, reactions, presence, and read receipts.

XP and content quality

The post composer shows XP values for categories. Education and News receive the strongest XP weight because AgroLink is designed to reward posts that help people learn, share practical updates, and make better local decisions. This gives users a reason to post useful farming knowledge, business notices, safety updates, market news, and learning content instead of only posting for attention. The feed can then become a community knowledge space, not just a place to scroll.

User experience

The frontend exposes social features through the Home Feed, Profile, Friends, Messages, Notifications, and shared shell navigation. The visual style is dashboard-like: dense, practical, and focused on repeated use.

COMMERCE

Business, Marketplace, Cart, and Orders

The business side lets farmers and small businesses publish products while buyers purchase directly from them. It connects business pages, products, cart persistence, checkout, order tracking, reviews, and analytics into one workflow.

Feature	What it does
Business pages	Business and farmer accounts can manage seller pages and public seller profiles.
Products	Sellers publish products with category, stock, availability, images, and delivery method.
Marketplace	Buyers browse products, search/filter, inspect details, add to cart, and navigate to sellers.
Cart	Buyers update quantities, view totals, remove items, clear cart, and proceed to checkout.
Orders	Checkout creates persisted backend orders. Buyers track orders and sellers update status.
Reviews	Users can leave public website or business-related reviews that support trust and discovery.
Analytics	Business and platform insights help sellers and admins understand product and order activity.

Cart UI update

The cart page follows the AgroLink yellow, green, and teal palette. Quantity changes stay in place, order summary areas are easier to scan, and the page visually matches the marketplace and order screens.

Order flow

- User adds products from marketplace to cart.
- Cart loads persisted backend cart rows.
- User reviews quantities and delivery methods.
- Checkout creates order records and clears the cart.
- Orders page displays buyer order history or seller received orders depending on role.

AgroLink Hub commerce features

PLATFORM TOOLS

Support, Calendar, Admin, and Chatbot Assistant

Support Center

Users can create support questions and view admin responses. Admin support routes allow management and replies.

Calendar

The calendar module supports events, reminders, farm tasks, business planning, birthdays, meetings, and visibility options.

Admin Command Center

Admin routes cover user groups, business users, farmers, creators, admins, moderation reports, and support ticket workflows.

Chatbot assistant

The assistant uses a knowledge base table named `concepts`. Seed data covers signup, login, marketplace, selling, posts, stories, orders, cart, reviews, support, calendar, chat, notifications, roles, and security.

Chatbot knowledge service

The backend chatbot service normalizes user messages, removes common stop words, compares query tokens against concept topics, keywords, and descriptions, and uses fuzzy scoring from Apache Commons Text. It returns a matching answer when the score is strong enough and a guided fallback when it cannot find a close match.

Landing assistant UI

The landing assistant panel has been enlarged and given more message space so messages no longer feel cramped. Prompt buttons and input layout are responsive for desktop and mobile.

Support and assistant value

These tools make the project easier to understand and use. Users can ask questions, open support tickets, and receive structured answers without depending only on manual navigation.

AgroLink Hub platform support features

OPERATIONS

Build, Testing, Deployment, and Final Notes

Verification notes

- Frontend production build was run with `npm run build`.
- The landing page, marketplace, cart, order, auth, and notification updates are documented in the repository README.
- The report PDF is generated from this HTML source so the editable report and exported report stay aligned.
- Backend API and database verification should be run with PostgreSQL credentials available in the local environment.

Known build warning

Vite reports that some chunks are larger than 500 kB after minification. This is a performance warning, not a build failure. It is documented here as a release note so the final handover clearly separates warnings from broken behavior.

Deployment checklist

- Set production PostgreSQL `DB_URL`, `DB_USERNAME`, and `DB_PASSWORD`.
- Set a strong `JWT_SECRET`; do not deploy with an empty secret.
- Configure mail credentials for OTP and password reset emails.
- Configure Google OAuth2 client ID, secret, and redirect URI.
- Set `COOKIE_SECURE=true` and appropriate cookie domain/same-site settings for HTTPS production.
- Restrict CORS origins to trusted production frontend domains.
- Use persistent upload storage for media files.
- Run backend and frontend builds before release.

Final closure statement

This report closes the project as a complete full-stack academic and product demonstration. The documentation reflects the implemented scope: social feed, XP-guided post categories, marketplace, cart, orders, business pages, chat with read ticks, notifications, calendar, support, analytics, admin workflows, and assistant support.

> Final summary: AgroLink Hub is a full-stack social commerce system for community engagement and direct selling. It combines authentication, role-based workspaces, social media, marketplace, cart, orders, chat, notifications, support, calendar, analytics, admin moderation, and a knowledge-base assistant.

End of AgroLink Hub project report